

Documentación Técnica

Módulo l10n_do_ecf_accounting

Facturación Electrónica RD — Odoo 19.0

ESPECIFICACIÓN TÉCNICA DE SOFTWARE

Tabla de Contenido

- 1. Información General del Módulo
- 2. Estructura de Directorios
- 3. Modelos Principales
- 4. Composición del Payload eCF
- 5. Flujo de Envío a DigiFact
- 6. Ambientes Sandbox y Producción
- 7. Tipos de Comprobantes Electrónicos
- 8. Datos Maestros y Cron Jobs
- 9. Reglas de Negocio Críticas
- 10. Zona Horaria y Redondeo Fiscal
- 11. Registro de Eventos (ecf.logger)
- 12. Pruebas y Validación Manual

1. Información General del Módulo

El presente documento constituye la especificación técnica detallada del módulo **l10n_do_ecf_accounting**, desarrollado para extender la localización dominicana de Odoo 19 con las capacidades necesarias para dar cumplimiento a la norma 32-23 de la Dirección General de Impuestos Internos (DGII), correspondiente a Facturación Electrónica. El módulo orquesta la construcción del documento fiscal electrónico (eCF), su firma y envío al proveedor de servicios DigiFact, así como el procesamiento de la respuesta emitida por la DGII.

1.1 Atributos del Módulo

Atributo	Valor
Nombre técnico	l10n_do_ecf_accounting
Nombre visible	Electronic Fiscal Accounting (Rep. Dominicana)
Versión	19.0.1.0.0
Autor original	Erick Cuesto
Categoría	Localization
Licencia	LGPL-3
Aplicación	No (módulo auxiliar)
Auto-instalación	No

1.2 Dependencias del Módulo

El módulo declara la siguiente dependencia directa en su archivo `__manifest__.py`:

Dependencia	Descripción
l10n_do_accounting	Provee los tipos de documento fiscal dominicanos (NCF y e-NCF), la gestión de lotes de comprobantes, el prefijo de secuencia fiscal (l10n_do_sequence_prefix) y los campos base de facturación fiscal.

1.3 Librerías de Terceros Requeridas

- **requests** — comunicación HTTPS con DigiFact (obtención de token y envío del payload).
- **qrcode + Pillow** — generación del código QR de validación DGII.
- **pytz** — resolución de zona horaria local (America/Santo_Domingo).
- **markupsafe** — composición segura del mensaje de chatter tras aceptación.

1.4 Resumen Funcional

- Envío de facturas electrónicas a DGII vía DigiFact al confirmarse el documento.
- Construcción del payload JSON eCF conforme a la norma 32-23 (Header, Seller, Buyer, Items, Totals, Payments, AdditionalDocumentInfo).
- Soporte para tipos E31, E32, E33, E34, E41, E43, E44, E45, E46 y E47.
- Gestión dual de credenciales y URLs: Sandbox (pruebas) y Producción (restringida al grupo del sistema).

- Renovación automática del token de autenticación (cron cada 25 días).
- Generación y almacenamiento del código QR de validación DGII por factura.
- Bloqueo de retorno a borrador de facturas electrónicas ya emitidas.
- Registro centralizado de errores de envío en el modelo ecf.logger con visibilidad en el chatter.

2. Estructura de Directorios

El módulo sigue la organización convencional de Odoo 19:

```
l10n_do_ecf_accounting/
■■■■ __init__.py
■■■■ __manifest__.py
■■■■ data/
■ ■■■■ ir_cron_data.xml          # Cron renovación token eCF (25 días)
■ ■■■■ neutralize.sql           # Limpieza credenciales al duplicar BD
■■■■ models/
■ ■■■■ __init__.py
■ ■■■■ account_move.py         # Lógica principal de envío/recepción eCF
■ ■■■■ account_move_line.py    # Tipo de impuesto facturación (compute)
■ ■■■■ account_payment_term.py # Tipo de pago (Contado / Crédito)
■ ■■■■ account_tax.py          # Indicadores de facturación ITBIS y adicionales
■ ■■■■ ecf_composition.py      # Clases constructoras del payload JSON
■ ■■■■ ecf_logger.py           # Modelo de log de errores de envío
■ ■■■■ res_company.py          # Ambiente Sandbox/Producción + credenciales
■ ■■■■ res_partner.py          # Flag "Es Receptor Electrónico?"
■ ■■■■ uom_uom.py              # Código ECF de unidad de medida
■■■■ report/
■ ■■■■ report_invoice.xml      # Plantilla factura con bloque eCF
■■■■ security/
■ ■■■■ ir.model.access.csv     # ACL modelo ecf.logger
■■■■ views/
■■■■ account_move_views.xml
■■■■ account_payment_term_views.xml
■■■■ account_tax_views.xml
■■■■ res_company_views.xml
■■■■ res_partner_views.xml
■■■■ uom_uom_views.xml
```

3. Modelos Principales

3.1 res.company — res_company.py

Centraliza la configuración de conexión con el proveedor DigiFact. Se adoptó un diseño dual sandbox/producción con selector explícito, de modo que el módulo conserve ambos sets de credenciales y pueda alternar sin sobrescribir.

Campo	Tipo	Descripción
l10n_do_ecf_environment	Selection	Ambiente activo: sandbox production. Por defecto sandbox.
l10n_do_ecf_phone	Char	Teléfono del emisor declarado en el eCF.
l10n_do_ecf_commercial_name	Char	Nombre comercial / ID de sucursal.

l10n_do_url_generate_token_{env}	Char	Endpoint para solicitar token. Un campo por ambiente.
l10n_do_username_generate_token_{env}	Char	Usuario de autenticación por ambiente.
l10n_do_password_generate_token_{env}	Char	Contraseña de autenticación por ambiente.
l10n_do_token_{env}	Text	Token vigente (readonly). Se rota vía cron.
l10n_do_token_expires_{env}	Char	Fecha/hora de expiración reportada por DigiFact.
l10n_do_url_api_{env}	Char	Endpoint de certificación del eCF.
l10n_do_timeout_url_api_request	Integer	Timeout en segundos para requests a DigiFact (default 60).

Restricción de acceso: todos los campos con sufijo `_production` están protegidos con `groups="base.group_system"`, de modo que solo un administrador del sistema pueda visualizarlos o modificarlos.

3.1.1 Métodos clave de `res.company`

- `_ecf_env_suffix()` — devuelve `"_sandbox"` o `"_production"` según el ambiente activo.
- `_get_active_ecf_creds()` — retorna un dict unificado con las credenciales y URLs del ambiente activo. Punto único de lectura.
- `_set_active_ecf_token(token, expires)` — escribe el token recién recibido en el ambiente activo.
- `l10n_do_generate_token()` — solicita un token fresco a DigiFact y lo persiste mediante el helper anterior.
- `_update_ecf_token()` — método de cron (`@api.model`) que itera todas las compañías, renovando tokens y aislando fallos por compañía.

3.2 `account.move` — `account_move.py`

Modelo principal. Extiende `account.move` con el ciclo de vida completo del eCF: estado DGII, identificador de rastreo, código de seguridad, fecha de firma, URL de validación, payload enviado y código QR. Los campos más relevantes:

Campo	Propósito
l10n_do_ecf_type	Integer computado a partir del <code>l10n_do_sequence_prefix</code> (E31 → 31).
l10n_do_ecf_status	Selection: 0 sin enviar, 1 aceptado, 2 rechazado, 3 en proceso, 4 aceptado condicional.
l10n_do_ecf_track_id	TrackID devuelto por DGII (<code>authNumber</code>).
l10n_do_ecf_security_code_from_dgii	Código de seguridad de 6 dígitos emitido por DGII.
l10n_do_ecf_signed_date_from_dgii	Fecha de firma reportada por DGII (sólo E32).
l10n_do_dgii_url_invoice_validate	URL pública de validación de la factura.
l10n_do_ecf_qr_code	Imagen PNG con el QR generado desde la URL de validación.
l10n_do_ecf_payload	Texto con el JSON enviado (auditoría).
l10n_do_ecf_message	Mensaje textual de DGII (motivo).
resend_ecf	Flag que marca facturas con envío fallido para reintento manual.
total_ritbis / total_isr	Acumuladores de retenciones durante la construcción del payload.

ecf_logger_ids	One2many hacia ecf.logger con los errores de envío.
----------------	---

3.2.1 Métodos del ciclo de envío

- **action_post()** — override que dispara action_send_ws() cuando la factura es confirmada desde el backend (context confirm_backend=True). Itera el recordset.
- **button_cancel()** — para facturas eCF ya emitidas cancela directamente sin devolver a borrador (los e-NCF aceptados son inmutables para DGII).
- **button_draft()** — impide resetear a borrador una factura eCF emitida (UserError).
- **action_send_ws()** — verifica que la factura sea elegible (tipo de movimiento + prefijo E*) y llama a build_invoice(). Registra el error en ecf.logger y chatter si falla, marcando resend_ecf.
- **build_invoice()** — construye el payload completo: Header → Seller → Buyer → Items → Totals → Payments → AdditionalDocumentInfo. Invoca _certificate_ecf().
- **_certificate_ecf(json_payload)** — realiza el POST a la URL de certificación del ambiente activo con el token vigente. El payload se persiste en l10n_do_ecf_payload antes del envío.
- **process_ecf_json_response(data_response)** — actualiza campos según la respuesta DGII, crea el adjunto XML y publica mensaje en chatter si la aceptación fue exitosa.
- **make_qr_code(url)** — genera la imagen QR en memoria con qrcode+Pillow y la guarda como Image.
- **_get_issued_datetime()** — compone la fecha-hora de emisión usando la zona America/Santo_Domingo (requisito fiscal).
- **_ecf_format_amount(value)** — helper introducido en Ola 3 que aplica currency_id.round() y formatea a 2 decimales antes de enviar montos a DGII.

3.3 account.move.line — account_move_line.py

Añade un campo computado l10n_do_ecf_tax_type (Integer no almacenado) que clasifica la línea como ITBIS tasa 1, 2 o 3 en función del grupo de impuesto vinculado. Se usa durante la construcción del bloque Items del payload.

3.4 account.tax — account_tax.py

- **l10n_do_billing_type_indicator** — Selection: 0 No Facturable, 1 ITBIS 18%, 2 ITBIS 16%, 3 ITBIS 0%, 4 Exento.
- **is_additional_tax** — Boolean para marcar impuestos adicionales (propina, telecomunicaciones, etc.).
- **l10n_do_billing_type_indicator_additional** — Selection con los 5 códigos adicionales (001-005).
- **purchase_tax_type** — Selection: ritbis | isr. Identifica retenciones en facturas de compra para el cálculo de MontoITBISRetenido / MontoISRRetenido.
- **get_code_billing_type_indicator()** / **get_code_billing_type_indicator_additional()** — mapean selección a código eCF (ITBIS1, ITBIS2, 001, etc.).

3.5 account.payment.term — account_payment_term.py

Agrega `l10n_do_ecf_payment_type` (Selection 1=Contado / 2=Crédito, default 2). Determina el tipo de pago declarado en el eCF cuando la factura se emite a crédito.

3.6 `res.partner` — `res_partner.py`

Añade `l10n_do_is_ecf_receptor` (Boolean) para marcar contactos habilitados como receptores electrónicos. La lógica de negocio consume este flag al construir el bloque Buyer del payload.

3.7 `uom.uom` — `uom_uom.py`

Incorpora el campo `l10n_do_ecf_uom` (Char) que permite mapear la unidad de medida de Odoo al código alfanumérico DGII (p. ej. "43" como valor por defecto para "Unidad").

3.8 `ecf.logger` — `ecf_logger.py`

Modelo dedicado a auditoría de errores de envío. Una línea por intento fallido, con referencia al `account.move`. El modelo es Read/Write/Create pero no Unlink (control de acceso en `security/ir.model.access.csv`).

4. Composición del Payload eCF

El archivo `models/ecf_composition.py` concentra las clases constructoras del JSON que será transmitido a DGII a través de DigiFact. Cada clase representa un bloque de la estructura eCF y expone métodos `set_*`() para poblarla de forma incremental. La serialización final se resuelve con `repr_json()`.

Clase	Sección del eCF
Header	Encabezado: fecha de emisión, tipo de eCF, datos de emisión adicionales.
Seller	Datos del emisor: RNC, razón social, nombre comercial, teléfono, dirección.
Buyer	Datos del comprador: RNC/Cédula, razón social, dirección, correo, indicador de envío electrónico.
Item	Bloque por cada línea de factura: tipo, descripción, cantidad, precio, impuestos adicionales, información complementaria.
Totals	Totales y bloque <code>TotalTaxes</code> (por código ITBIS). Incluye <code>TotalITBISRetenido</code> / <code>TotalISRRetencion</code> cuando aplica.
Payment	Modo(s) de pago: código (efectivo, crédito, tarjeta, etc.) y monto.
AdditionalDocumentInfo	Información adicional del documento (notas de referencia, motivos).
TokenBuilder	Arma el payload de autenticación requerido por DigiFact para generar token.
EcfJsonBuilder	Orquestador raíz: compone todas las secciones y emite el JSON definitivo.

Criterio de redondeo fiscal: desde la Ola 3 todos los montos emitidos se filtran por `currency_id.round()` y se formatean a 2 decimales antes de ingresar al JSON, evitando inconsistencias como "15.000000000000002" al momento de serializar.

5. Flujo de Envío a DigiFact

El envío al proveedor es disparado automáticamente por la confirmación de la factura en el backend. El flujo completo, paso a paso:

- 1. Usuario confirma factura → `action_post()` con `context confirm_backend=True`.

- 2. Por cada move del recordset se invoca `action_send_ws()`.
- 3. `action_send_ws()` valida estado == posted, `is_ecf_invoice` y prefijo de secuencia compatible con el `move_type`.
- 4. Si es elegible, llama a `build_invoice()` dentro de un try/except.
- 5. `build_invoice()` arma las 7 secciones del payload y ejecuta `_certificate_ecf(json_payload)`.
- 6. `_certificate_ecf()` lee credenciales del ambiente activo, persiste el payload y hace POST HTTPS.
- 7. `process_ecf_json_response()` mapea la respuesta: actualiza status, trackId, mensaje, firma, URL de validación y crea adjunto XML.
- 8. Si la respuesta incluye URL pública, `update_ecf_attrs()` extrae el código de seguridad y genera el QR vía `make_qr_code()`.
- 9. En caso de error: la excepción se captura, se marca `resend_ecf=True`, se crea `ecf.logger` y se publica el error en chatter.

5.1 Criterios de Elegibilidad de Envío

Move Type	Prefijos Compatibles
out_invoice / out_refund	E31, E32, E33, E34, E44, E45
in_invoice / in_refund	E41, E43

6. Ambientes Sandbox y Producción

DigiFact expone dos entornos independientes. El módulo provee una UI unificada para gestionar ambos y un selector `l10n_do_ecf_environment` que determina cuál se usa en cada operación. Esto permite:

- Mantener credenciales de prueba y producción simultáneamente sin sobrescribir.
- Conmutar el comportamiento del módulo con un solo cambio (sin reconfigurar URLs).
- Restringir los campos de producción al grupo del sistema (ocultos para usuarios estándar).
- Aislar las credenciales al duplicar BDs: el script `data/neutralize.sql` limpia ambos sets y fuerza sandbox.

6.1 Flujo de Generación de Token

- El usuario (o el cron) invoca `l10n_do_generate_token()`.
- Se obtienen las credenciales con `_get_active_ecf_creds()`.
- Si falta alguna credencial, se eleva `ValidationError` con el nombre del ambiente.
- `TokenBuilder` arma el payload y se envía POST a `url_generate_token`.
- La respuesta JSON trae Token + `expira_en`. Se persisten vía `_set_active_ecf_token()`.
- Cron `ir_cron_res_company_ecf_token` recorre todas las compañías cada 25 días. Cualquier fallo individual se registra en log sin abortar el cron.

7. Tipos de Comprobantes Electrónicos Soportados

Código	Descripción
E31	Factura de Crédito Fiscal Electrónica.
E32	Factura Electrónica de Consumo.
E33	Nota de Débito Electrónica.
E34	Nota de Crédito Electrónica.
E41	Compras Electrónicas.
E43	Gastos Menores Electrónicos.
E44	Regímenes Especiales Electrónicos.
E45	Gubernamental Electrónica.
E46	Exportaciones Electrónicas.
E47	Pagos al Exterior Electrónicos.

8. Datos Maestros y Cron Jobs

8.1 data/ir_cron_data.xml

Define el cron **Update eCF Token** que invoca `res.company._update_ecf_token()`. Intervalo: 25 días. Priority: 8. Modelo: `res.company`.

8.2 data/neutralize.sql

Script de neutralización ejecutado al duplicar o clonar la base de datos. Limpia las credenciales de sandbox y producción, fuerza el ambiente a sandbox y evita que una BD clonada dispare envíos accidentales a la DGII.

9. Reglas de Negocio Críticas

- **No se puede devolver a borrador una factura eCF emitida:** `button_draft` lanza `UserError`. Motivo: el e-NCF aceptado es inmutable ante DGII.
- **Cancelación directa para eCF emitidos:** `button_cancel` omite el paso por draft y pasa directamente a cancel, removiendo reconciliaciones y marcando pagos como canceled.
- **Facturas con envío fallido reintentan manualmente:** el flag `resend_ecf` identifica los movimientos pendientes. Los usuarios pueden relanzar desde la interfaz.
- **El payload enviado se guarda en la factura:** `l10n_do_ecf_payload` preserva el JSON textual que salió hacia DigiFact; útil para auditoría y depuración.
- **El QR se genera a partir de la URL pública:** `make_qr_code()` decodifica la URL retornada por DGII, extrae el parámetro `codigoseguridad` y compone el código QR con `qrcode` + `Pillow`.
- **Retenciones compradas (R-ITBIS / ISR):** se identifican por `tax.purchase_tax_type` y se acumulan en los campos `total_ritbis` / `total_isr` al construir el payload. Cada línea se redondea y formatea a 2 decimales.

10. Zona Horaria y Redondeo Fiscal

Zona horaria. La hora de emisión enviada a DGII se construye desde `datetime.now(pytz.timezone("America/Santo_Domingo"))` y se concatena con la fecha de factura como `YYYY-MM-DDThh:mm:ss`. Esto garantiza que el sello temporal sea consistente con la hora local RD, con independencia del TZ del servidor Odoo.

Redondeo fiscal. Toda cantidad monetaria que se serializa al payload pasa por `_ecf_format_amount()` o por `currency_id.round()`. Puntos de aplicación:

- `Item.TotalItems` (precio subtotal de línea).
- `TotalTaxes[*].TaxableAmount` y `TotalTaxes[*].Amount`.
- `additional_taxes[*].Amount` dentro de cada `Item`.
- `MontoITBISRetenido`, `MontoISRRetenido` en `AdditionalInfo` de línea.
- `TotalITBISRetenido`, `TotalISRRetencion` en `AdditionalInfo` de Totals.
- `GrandTotal.InvoiceTotal` (formateado a 2 decimales dentro de `ecf_composition.py`).

11. Registro de Eventos (ecf.logger)

Modelo transient-less dedicado a persistir errores de envío. Estructura:

Campo	Tipo	Descripción
<code>name</code>	Char	Etiqueta corta del evento (e.g. "Error").
<code>log</code>	Text	Trazabilidad del error (mensaje completo).
<code>move_id</code>	Many2one	Factura asociada (<code>account.move</code>).

El ACL permite lectura, escritura y creación a todos los usuarios con acceso a `account.move`, pero deshabilita `unlink`. Los registros se exponen en la factura vía `One2many` (`ecf_logger_ids`) para facilitar el diagnóstico.

12. Pruebas y Validación Manual

El módulo no incluye una suite de tests automatizados en la versión 19.0.1.0.0. Los puntos de validación recomendados para el QA manual son:

- Instalar el módulo sobre una BD con `l10n_do_accounting`: debe cargar sin errores (verificado: 99 módulos en ~5s).
- Configurar compañía en ambiente Sandbox con credenciales DigiFact válidas y generar token con el botón.
- Emitir una factura de venta con prefijo E31/E32 sobre un cliente con `l10n_do_is_ecf_receptor=True`.
- Verificar que `l10n_do_ecf_status` pase a "1" (Aceptado) y que el QR y el adjunto XML se generen.
- Inducir un error (credencial inválida) y verificar que `ecf_logger_ids` se pueble y que `resend_ecf` quede en `True`.
- Intentar `button_draft` sobre una factura eCF: debe lanzar `UserError`.

- Conmutar ambiente a Producción y validar que los campos queden ocultos para un usuario no-admin.

Manual elaborado por: Erick Daniel Cuesto Mendez

Versión 1.0 — Abril 2026 — Documento N.º ECM-USR-001

Confidencial